

SUM(Sales)
OVER(ORDER BY Month
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)

	Month	Sales	Result
UNBOUNDED PRECEDING	Jan	20	135
	Feb	10	135
Current Row	Mar	30	135
	Apr	5	135
UNBOUNDED FOLLOWING	Jun	70	135



Object Explorer: Connect to DESKTOP-8488QBU\SQLEXPRESS

```

SELECT
  OrderID,
  OrderDate,
  OrderStatus,
  Sales,
  SUM(Sales) OVER (PARTITION BY OrderStatus ORDER BY OrderDate
  ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING) TotalSales
FROM Sales.Orders
  
```

	OrderID	OrderDate	OrderStatus	Sales	TotalSales
1	1	2025-01-01	Delivered	10	55
2	3	2025-01-10	Delivered	20	95
3	5	2025-02-01	Delivered	25	105
4	6	2025-02-05	Delivered	50	80
5	7	2025-02-15	Delivered	30	30
6	2	2025-01-05	Shipped	15	165
7	4	2025-01-20	Shipped	60	170
8	8	2025-02-18	Shipped	90	170
9	9	2025-03-10	Shipped	20	80
10	10	2025-03-15	Shipped	60	60

#1 WINDOW

Video frame with social media icons (like, share, comment) at the bottom right.

Object Explorer

Connect +

DESKTOP-8488QBU\SQLEXPRESS

Databases

Security

Server Objects

Replication

Management

XEvent Profiler

--vs6A1D.sql DES_8QBU\Youtube (52)

```

SELECT
  OrderID,
  OrderDate,
  OrderStatus,
  Sales,
  SUM(Sales) OVER (PARTITION BY OrderStatus ORDER BY OrderDate
    ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING) TotalSales
FROM Sales.Orders
  
```

#1 WINDOW

	OrderID	OrderDate	OrderStatus	Sales	TotalSales
1	1	2025-01-01	Delivered	10	55
2	3	2025-01-10	Delivered	20	95
3	5	2025-02-01	Delivered	25	105
4	6	2025-02-05	Delivered	50	80
5	7	2025-02-15	Delivered	30	30
6	2	2025-01-05	Shipped	15	165
7	4	2025-01-20	Shipped	60	170
8	8	2025-02-18	Shipped	90	170
9	9	2025-03-10	Shipped	20	80
10	10	2025-03-15	Shipped	60	60



Object Explorer

Connect +

DESKTOP-8488QBU\SQLEXPRESS

Databases

Security

Server Objects

Replication

Management

XEvent Profiler

--vs6A1D.sql DES_8QBU\Youtube (52)

```

SELECT
  OrderID,
  OrderDate,
  OrderStatus,
  Sales,
  SUM(Sales) OVER (PARTITION BY OrderStatus ORDER BY OrderDate
    ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING) TotalSales
FROM Sales.Orders
  
```

#1 WINDOW

	OrderID	OrderDate	OrderStatus	Sales	TotalSales
1	1	2025-01-01	Delivered	10	55
2	3	2025-01-10	Delivered	20	95
3	5	2025-02-01	Delivered	25	105
4	6	2025-02-05	Delivered	50	80
5	7	2025-02-15	Delivered	30	30
6	2	2025-01-05	Shipped	15	165
7	4	2025-01-20	Shipped	60	170
8	8	2025-02-18	Shipped	90	170
9	9	2025-03-10	Shipped	20	80
10	10	2025-03-15	Shipped	60	60



In below image two records are outside window frame calculate inside window frame only.

Object Explorer

Connect

DESKTOP-8488QBU\SQLEXPRESS

Databases

Security

Server Objects

Replication

Management

XE Event Profiler

Execute

--vs6A1D.sql - DES_8QBU\Youtube (52) --

SELECT

OrderID,

OrderDate,

OrderStatus,

Sales,

SUM(Sales) OVER (PARTITION BY OrderStatus ORDER BY OrderDate

ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING) TotalSales

FROM Sales.Orders

#1 WINDOW

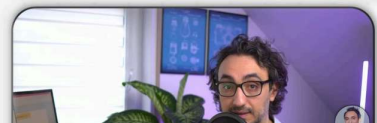
	OrderID	OrderDate	OrderStatus	Sales	TotalSales
1	1	2025-01-01	Delivered	10	55
2	3	2025-01-10	Delivered	20	95
	5	2025-02-01	Delivered	25	105
4	6	2025-02-05	Delivered	50	80
5	7	2025-02-15	Delivered	30	30
6	2	2025-01-05	Shipped	15	165
7	4	2025-01-20	Shipped	60	170
8	8	2025-02-18	Shipped	90	170
9	9	2025-03-10	Shipped	20	80
10	10	2025-03-15	Shipped	60	60

COMPACT FRAME

For only PRECEDING, the CURRENT ROW can be skipped

NORMAL FORM ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING

SHORT FORM ROWS 2 FOLLOWING



~vs6A1D.sql - DES...8QBU\Youtube (52))

5

SELECT

OrderID,

OrderDate,

OrderStatus,

Sales,

SUM(Sales) OVER (PARTITION BY OrderStatus ORDER BY OrderDate

ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) TotalSales

FROM Sales.Orders

OrderID	OrderDate	OrderStatus	Sales	TotalSales
1	2025-01-01	Delivered	10	10
3	2025-01-10	Delivered	20	30
5	2025-02-01	Delivered	25	55
6	2025-02-05	Delivered	50	95
7	2025-02-15	Delivered	30	105
2	2025-01-05	Shipped	15	15
4	2025-01-20	Shipped	60	75
8	2025-02-18	Shipped	90	165
9	2025-03-10	Shipped	20	170
10	2025-03-15	Shipped	60	170



Shortcut -

~vs6A1D.sql - DES...8QBU\Youtube (52)*

SELECT

OrderID,

OrderDate,

OrderStatus,

Sales,

SUM(Sales) OVER (PARTITION BY OrderStatus ORDER BY OrderDate

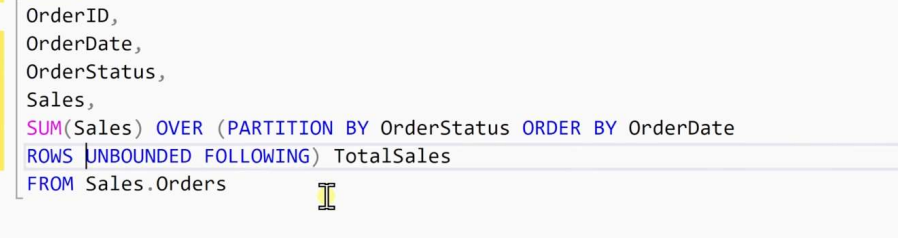
ROWS 2 PRECEDING) TotalSales

FROM Sales.Orders

	OrderID	OrderDate	OrderStatus	Sales	TotalSales
1	1	2025-01-01	Delivered	10	10
2	3	2025-01-10	Delivered	20	30
3	5	2025-02-01	Delivered	25	55
4	6	2025-02-05	Delivered	50	95
5	7	2025-02-15	Delivered	30	105
6	2	2025-01-05	Shipped	15	15
7	4	2025-01-20	Shipped	60	75
8	8	2025-02-18	Shipped	90	165
9	9	2025-03-10	Shipped	20	170
10	10	2025-03-15	Shipped	60	170

```
SELECT
  OrderID,
  OrderDate,
  OrderStatus,
  Sales,
  SUM(Sales) OVER (PARTITION BY OrderStatus ORDER BY OrderDate
    ROWS UNBOUNDED PRECEDING) TotalSales
FROM Sales.Orders
```

	OrderID	OrderDate	OrderStatus	Sales	TotalSales
1	1	2025-01-01	Delivered	10	10
2	3	2025-01-10	Delivered	20	30
3	5	2025-02-01	Delivered	25	55
4	6	2025-02-05	Delivered	50	105
5	7	2025-02-15	Delivered	30	135
6	2	2025-01-05	Shipped	15	15
7	4	2025-01-20	Shipped	60	75
8	8	2025-02-18	Shipped	90	165
9	9	2025-03-10	Shipped	20	185
10	10	2025-03-15	Shipped	60	245



```
--vs6A1D.sql - DES...8QBU\Youtube (52) - X
SQL EXPRESS
SELECT
  OrderID,
  OrderDate,
  OrderStatus,
  Sales,
  SUM(Sales) OVER (PARTITION BY OrderStatus ORDER BY OrderDate
  ROWS UNBOUNDED FOLLOWING) TotalSales
FROM Sales.Orders
```

Msg 102, Level 15, State 1, Line 7
Incorrect syntax near 'FOLLOWING'.

Completion time: 2024-02-27T17:23:15.0786410

Execute

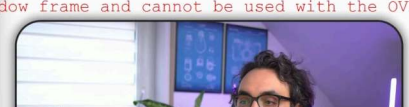
~vs6A1D.sql - DES...BQBU\Youtube (52)*

SQL Server Enterprise Edition

Msg 4193, Level 16, State 4, Line 1
'BETWEEN ... FOLLOWING AND CURRENT ROW' is not supported in this context.

Completion time: 2024-02-27T17:23:23.951057

Row frame and cannot be used with the OVER clause.



DEFAULT FRAME

SQL uses **Default Frame**, if **ORDER BY** is used **without FRAME**

SQL Server Enterprise Edition

~vs6A1D.sql - DES...BQBU\Youtube (52)*

SELECT
OrderID,
OrderDate,
OrderStatus,
Sales,
SUM(Sales) OVER (PARTITION BY OrderStatus ORDER BY OrderDate) TotalSales
FROM Sales.Orders

	OrderID	OrderDate	OrderStatus	Sales	TotalSales
1	1	2025-01-01	Delivered	10	10
2	3	2025-01-10	Delivered	20	30
3	5	2025-02-01	Delivered	25	55
4	6	2025-02-05	Delivered	50	105
5	7	2025-02-15	Delivered	30	135
6	2	2025-01-05	Shipped	15	15
7	4	2025-01-20	Shipped	60	75
8	8	2025-02-18	Shipped	20	105



SQL Query:

```
SELECT
OrderID,
OrderDate,
OrderStatus,
Sales,
SUM(Sales) OVER (PARTITION BY OrderStatus ORDER BY OrderDate
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) TotalSales
FROM Sales.Orders
```

	OrderID	OrderDate	OrderStatus	Sales	TotalSales
1	1	2025-01-01	Delivered	10	10
2	3	2025-01-10	Delivered	20	30
3	5	2025-02-01	Delivered	25	55
4	6	2025-02-05	Delivered	50	105
5	7	2025-02-15	Delivered	30	135
6	2	2025-01-05	Shipped	15	15
7	4	2025-01-20	Shipped	60	75
8	8	2025-02-18	Shipped	90	165
9	9	2025-03-10	Shipped	20	185
10	10	2025-03-15	Shipped	60	245

Above is default frame
Default frame only working with order by
Or shortcut

SQL Query:

```
SELECT
OrderID,
OrderDate,
OrderStatus,
Sales,
SUM(Sales) OVER (PARTITION BY OrderStatus ORDER BY OrderDate
ROWS UNBOUNDED PRECEDING) TotalSales
FROM Sales.Orders
```

	OrderID	OrderDate	OrderStatus	Sales	TotalSales
1	1	2025-01-01	Delivered	10	10
2	3	2025-01-10	Delivered	20	30
3	5	2025-02-01	Delivered	25	55
4	6	2025-02-05	Delivered	50	105
5	7	2025-02-15	Delivered	30	135
6	2	2025-01-05	Shipped	15	15
7	4	2025-01-20	Shipped	60	75
8	8	2025-02-18	Shipped	90	165
9	9	2025-03-10	Shipped	20	185
10	10	2025-03-15	Shipped	60	245

```

SELECT
  OrderID,
  OrderDate,
  OrderStatus,
  Sales,
  SUM(Sales) OVER (PARTITION BY OrderStatus) TotalSales
FROM Sales.Orders

```

	OrderID	OrderDate	OrderStatus	Sales	TotalSales
1	1	2025-01-01	Delivered	10	135
2	3	2025-01-10	Delivered	20	135
3	5	2025-02-01	Delivered	25	135
4	6	2025-02-05	Delivered	50	135
5	7	2025-02-15	Delivered	30	135
6	8	2025-02-18	Shipped	90	245

No default frame without order clause

Limitations of window functions.

What you are not allowed to do while using window function



#1 RULE

Window functions can be used **ONLY**
in **SELECT** and **ORDER BY** Clauses

```
--vs7ED5.sql - DES...8QBU\Youtube (60)) * X --vs5021.sql - DES...8QBU\Youtube (53))  
SELECT  
    OrderID,  
    OrderDate,  
    OrderStatus,  
    Sales,  
    SUM(Sales) OVER (PARTITION BY OrderStatus) TotalSales  
FROM Sales.Orders
```

226 %

Results Messages

	OrderID	OrderDate	OrderStatus	Sales	TotalSales
1	1	2025-01-01	Delivered	10	135
2	3	2025-01-10	Delivered	20	135
3	5	2025-02-01	Delivered	25	135
4	6	2025-02-05	Delivered	50	135
5	7	2025-02-15	Delivered	30	135
6	8	2025-02-18	Shipped	90	245
7	9	2025-03-10	Shipped	20	245
8	10	2025-03-15	Shipped	60	245
9	4	2025-01-20	Shipped	60	245
10	2	2025-01-05	Shipped	15	245



Object Explorer

SQL Query Editor

~vs7ED5.sql - DES...8QBU\Youtube (60)) * x ~vs5021.sql - DES...8QBU\Youtube (53))

```
SELECT
    OrderID,
    OrderDate,
    OrderStatus,
    Sales,
    SUM(Sales) OVER (PARTITION BY OrderStatus) TotalSales
FROM Sales.Orders
ORDER BY SUM(Sales) OVER (PARTITION BY OrderStatus)
```

226 %

Results Messages

	OrderID	OrderDate	OrderStatus	Sales	TotalSales
1	1	2025-01-01	Delivered	10	135
2	3	2025-01-10	Delivered	20	135
3	5	2025-02-01	Delivered	25	135
4	6	2025-02-05	Delivered	50	135
5	7	2025-02-15	Delivered	30	135
6	8	2025-02-18	Shipped	90	245
7	9	2025-03-10	Shipped	20	245
8	10	2025-03-15	Shipped	60	245
9	4	2025-01-20	Shipped	60	245
10	2	2025-01-05	Shipped	15	245

Query executed successfully.

47:22 / 57:18 4 Rules of Window >

Object Explorer

SQL Query Editor

~vs7ED5.sql - DES...8QBU\Youtube (60)) * x ~vs5021.sql - DES...8QBU\Youtube (53))

```
SELECT
    OrderID,
    OrderDate,
    OrderStatus,
    Sales,
    SUM(Sales) OVER (PARTITION BY OrderStatus) TotalSales
FROM Sales.Orders
ORDER BY SUM(Sales) OVER (PARTITION BY OrderStatus) DESC
```

226 %

Results Messages

	OrderID	OrderDate	OrderStatus	Sales	TotalSales
1	8	2025-02-18	Shipped	90	245
2	9	2025-03-10	Shipped	20	245
3	10	2025-03-15	Shipped	60	245
4	4	2025-01-20	Shipped	60	245
5	2	2025-01-05	Shipped	15	245
6	1	2025-01-01	Delivered	10	135
7	3	2025-01-10	Delivered	20	135
8	5	2025-02-01	Delivered	25	135
9	6	2025-02-05	Delivered	50	135
10	7	2025-02-15	Delivered	30	135

NOTE

WINDOW Functions can't be used to filter Data

built-in function SUM(expression) RETURNS int

```
SELECT
    OrderID,
    OrderDate,
    OrderStatus,
    Sales,
    SUM(Sales) OVER (PARTITION BY OrderStatus) TotalSales
FROM Sales.Orders
WHERE SUM(Sales) OVER (PARTITION BY OrderStatus) > 100
```

226 %
Messages

Msg 4108, Level 15, State 1, Line 8
Windowed functions can only appear in the SELECT or ORDER BY clauses.

Completion time: 2024-03-02T13:59:08.4640664+01:00

```

SELECT
    OrderID,
    OrderDate,
    OrderStatus,
    Sales,
    SUM(Sales) OVER (PARTITION BY OrderStatus) TotalSales
FROM Sales.Orders
GROUP BY SUM(Sales) OVER (PARTITION BY OrderStatus)

```

226 %
 Messages
 Msg 4108, Level 15, State 1, Line 8
 Windowed functions can only appear in the SELECT or ORDER BY clauses.
 Completion time: 2024-03-02T13:59:50.9199323+01:00



#2 RULE

Nesting Window Functions is not allowed !

```

SELECT
    OrderID,
    OrderDate,
    OrderStatus,
    Sales,
    SUM(SUM(Sales) OVER (PARTITION BY OrderStatus)) OVER (PARTITION BY OrderStatus) TotalSales
FROM Sales.Orders

```

187 %
 Messages
 Msg 4109, Level 15, State 1, Line 6
 Windowed functions cannot be used in the context of another windowed function or aggregate.
 Completion time: 2024-03-02T14:01:09.1059043+01:00



#3 RULE

SQL execute WINDOW Functions after WHERE Clause

SQL TASK

Find the **total sales** for each order status,
only for **two products 101 and 102**

```
--vs7ED5.sql - DES...8QBU\Youtube (60)]* X --vs5021.sql - DES...8QBU\Youtube (53)]
SELECT
    OrderID,
    OrderDate,
    OrderStatus,
    ProductID,
    Sales,
    SUM(Sales) OVER (PARTITION BY OrderStatus) TotalSales
FROM Sales.Orders
WHERE ProductID IN (101,102)
```

187 %

Results Messages

	OrderID	OrderDate	OrderStatus	ProductID	Sales	TotalSales
1	3	2025-01-10	Delivered	101	20	60
2	7	2025-02-15	Delivered	102	30	60
3	1	2025-01-01	Delivered	101	10	60
4	2	2025-01-05	Shipped	102	15	185
5	8	2025-02-18	Shipped	101	90	185
6	9	2025-03-10	Shipped	101	20	185
7	10	2025-03-15	Shipped	102	60	185



#4 RULE

Window Function can be used together with GROUP BY
in the same query, ONLY if the same **columns** are used

NOTE

**Use GROUP BY
for simple Aggregations**

But when ask simple aggregation along with ranking then in such cases we used window function together group by

1. For total sales used group by - simple aggregation.

Half task solve now

--Rank Customers based on their total sales

```
SELECT  
    CustomerID,  
    SUM(Sales) TotalSales  
FROM Sales.Orders  
GROUP BY CustomerID
```

	CustomerID	TotalSales
1	1	110
2	2	55
3	3	125
4	4	90

Step 2. Add window function to rank the customers.

Rank function always required order by inside over clause

– whatever is in the

Anythink you are using in the group by should be part of the window function

Object Explorer

--Rank Customers based on their total sales

```

SELECT
    CustomerID,
    SUM(Sales) TotalSales,
    RANK() OVER(ORDER BY SUM(Sales) DESC) RankCustomers
FROM Sales.Orders
GROUP BY CustomerID

```

187 %

Results Messages

	CustomerID	TotalSales	RankCustomers
1	3	125	1
2	1	110	2
3	4	90	3
4	2	55	4

Object Explorer

--Rank Customers based on their total sales

```

SELECT
    CustomerID,
    SUM(Sales) TotalSales,
    RANK() OVER(ORDER BY SUM(Sales) DESC) RankCustomers
FROM Sales.Orders
GROUP BY CustomerID

```

187 %

Results Messages

	CustomerID	TotalSales	RankCustomers
1	3	125	1
2	1	110	2
3	4	90	3
4	2	55	4



```
--Rank Customers based on their total sales
SELECT
    CustomerID,
    SUM(Sales) TotalSales,
    RANK() OVER(ORDER BY Sales DESC) RankCustomers
FROM Sales.Orders
GROUP BY CustomerID
```

87 %
Messages
Msg 8120, Level 16, State 1, Line 5
Column 'Sales.Orders.Sales' is invalid in the select list because it is not contained in either an aggregate function



```
--Rank Customers based on their total sales
SELECT
    CustomerID,
    SUM(Sales) TotalSales,
    RANK() OVER(ORDER BY CustomerID DESC) RankCustomers
FROM Sales.Orders
GROUP BY CustomerID
```

17 %
Results Messages

	CustomerID	TotalSales	RankCustomers
1	4	90	1
2	3	125	2
3	2	55	3
4	1	110	4

SQL WINDOW FUNCTIONS

Performs calculations on subset of data without losing details

Window v/s Group By

- Window is more Powerfull & Dynamic than Group By.
- Data Analysis
 - Advanced → Window
 - Simple → Group By
- Use Group By + window in same Query, only if same Column used.

Components

Window Functions + Window Definition OVER



Rules

- Nesting is not allowed!
- Window Can be used only in SELECT and ORDER BY
- SQL executes window AFTER filtering data using WHERE